ENSA Khouribga

Calcul scientifique

Mini-projet Python

Algèbre linéaire

Matrices et systèmes linéaires

Déterminant – Inverse – Résolution de $AX = b$

Réalisé par : Akebbad Mohcine

Encadré par : Pr. Mouad Hakim

Année universitaire 2026–2027

Introduction

Ce mini-projet a pour objectif de mettre en pratique avec Python les notions d'algèbre linéaire à l'aide de NumPy. On étudie la création et la manipulation de matrices, le déterminant, l'inversibilité, la résolution et la vérification d'un système linéaire.
Dans le contexte de l'atelier, les quantités produites des produits $P1$, $P2$ et $P3$ sont x_1, x_2, x_3. Le système est :

$$\begin{cases} 2x_1 + x_2 + x_3 = 110, \\ x_1 + 3x_2 + 2x_3 = 160, \\ 3x_1 + 2x_2 + 4x_3 = 240. \end{cases}$$

Il s'écrit $AX = b$, avec

$$A = \begin{pmatrix} 2 & 1 & 1 \\ 1 & 3 & 2 \\ 3 & 2 & 4 \end{pmatrix}, \qquad b = \begin{pmatrix} 110 \\ 160 \\ 240 \end{pmatrix}.$$

A. Prise en main des matrices

On obtient :

$$\text{shape}(A) = (3, 3), \qquad \text{shape}(b) = (3, 1).$$

La transposée est :

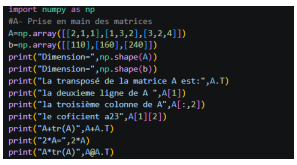
$$A^T = \begin{pmatrix} 2 & 1 & 3 \\ 1 & 3 & 2 \\ 1 & 2 & 4 \end{pmatrix}.$$

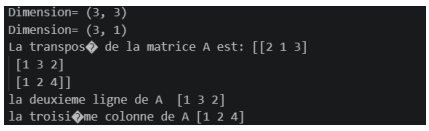
La deuxième ligne est $(1, 3, 2)$, la troisième colonne est $(1, 2, 4)^T$ et $a_{23} = 2$.
Les opérations donnent :

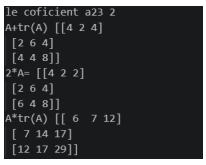
$$A + A^T = \begin{pmatrix} 4 & 2 & 4 \\ 2 & 6 & 4 \\ 4 & 4 & 8 \end{pmatrix}, \qquad 2A = \begin{pmatrix} 4 & 2 & 2 \\ 2 & 6 & 4 \\ 6 & 4 & 8 \end{pmatrix}.$$

$$AA^T = \begin{pmatrix} 6 & 7 & 12 \\ 7 & 14 & 17 \\ 12 & 17 & 29 \end{pmatrix}.$$

Différence entre `*` et `@` : `*` réalise une multiplication élément par élément alors que `@` réalise le produit matriciel. Ainsi, `A @ A.T` correspond à AA^T. Dans les messages `print`, le symbole `*` peut être utilisé pour indiquer le produit matriciel à l'utilisateur, mais l'opération exécutée est `@`.







B. Déterminant et inversibilité

Le déterminant est :

$$\det(A) = 11 \neq 0.$$

Donc A est inversible. On obtient :

$$A^{-1} \approx \begin{pmatrix} 0.7273 & -0.1818 & -0.0909 \\ 0.1818 & 0.4545 & -0.2727 \\ -0.6364 & -0.0909 & 0.4545 \end{pmatrix}.$$

La vérification AA^{-1} donne la matrice identité I_3 à la précision numérique près. Les valeurs très proches de zéro peuvent provenir des erreurs d'arrondi en virgule flottante.

```python
#B-Déterminant et inversibilité
det=np.linalg.det(A)
print("le determinant de A:",det)
if det==0:
    print("A est non invessible")
else:
    print("A est inverssible")
    Inv=np.linalg.inv(A)
    print("l'inverse de A est :",Inv)
    print("A*inv(A)",A@Inv)
```

```
le determinant de A: 11.000000000000002
A est inverssible
l'inverse de A est : [[ 0.72727273 -0.18181818 -0.09090909]
 [ 0.18181818  0.45454545 -0.27272727]
 [-0.63636364 -0.09090909  0.45454545]]
A*inv(A) [[ 1.00000000e+00 -1.38777878e-17  5.55111512e-17]
 [ 0.00000000e+00  1.00000000e+00  0.00000000e+00]
 [-4.44089210e-16 -5.55111512e-17  1.00000000e+00]]
```

C. Résolution du système linéaire

Puisque A est inversible :

$$X = A^{-1}b = \begin{pmatrix} 29.0909 \\ 27.2727 \\ 24.5455 \end{pmatrix}.$$

Ainsi :

$$x_1 \approx 29.09, \qquad x_2 \approx 27.27, \qquad x_3 \approx 24.55.$$

La fonction `np.linalg.solve(A,b)` donne le même résultat. La comparaison avec `np.allclose(X,Y)` donne `True`. La vérification donne :

$$AX \approx \begin{pmatrix} 110 \\ 160 \\ 240 \end{pmatrix} = b.$$

Le résidu $r = AX - b$ a une norme pratiquement nulle :

$$\|r\|_2 \approx 0.$$

Dans le contexte de l'atelier, x_1, x_2, x_3 représentent les quantités produites respectivement de $P1$, $P2$ et $P3$.

```python
# C-la résolution du système linéaire
X=Inv@b
print("la solution du système linéaire A*X=b est X=",X)
Y=np.linalg.solve(A,b)
print("la solutin par numpy est X=",Y)
print(np.allclose(X,Y))
AX=A@X
print("AX=",AX)
r=AX-b
print("le résidu",r)
print("la norme de r",np.linalg.norm(r))
```

```
la solution du syst?me lin?aire A*X=b est X= [[29.09090909]
 [27.27272727]
 [24.54545455]]
la solutin par numpy est X= [[29.09090909]
 [27.27272727]
 [24.54545455]]
True
AX= [[110.]
 [160.]
 [240.]]
```

```
le r?sidu [[ 0.00000000e+00]
 [-5.68434189e-14]
 [-5.68434189e-14]]
la norme de r 8.038873388460929e-14
```

D. Matrice singulière

On considère :

$$C = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 1 & 1 & 2 \end{pmatrix}, \qquad d = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}.$$

On obtient :

$$\det(C) = 0.$$

Donc C est non inversible (singulière). On remarque que :

$$L_2 = 2L_1.$$

Les lignes sont donc linéairement dépendantes. Cela explique $\det(C) = 0$ et l'absence d'une matrice inverse.

Les fonctions `np.linalg.inv(C)` et `np.linalg.solve(C,d)` provoquent une erreur pour cette matrice singulière. Le programme utilise `try/except` pour gérer ces erreurs.

```python
# D: Que se passe-t-il si la matrice est singulière ?
C=np.array([[1,2,3],[2,4,6],[1,1,2]])
d=np.array([[1],[2],[3]])
detC=np.linalg.det(C)
print("det(C)=",detC)
print("Alors la matrice est non inversible")
try:
    print("l'invese de C est:",np.linalg.inv(C))
except np.linalg.LinAlgError:
    print("il n'y pas d'invese pur ce matrice ")
try:
    print("la solution deCX=d est:",np.linalg.solve(C,d))
except:
    print("il est impossible de résoudre ce système")

print("on observe que la deuxième ligne est deux fois de la première L2=2L1 alors C est singulière ce qui explique le résultat")
```

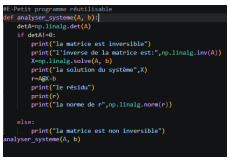
```
det(C)= 0.0
Alors la matrice est non inversible
il n'y pas d'invese pur ce matrice
il est impossible de r?soudre ce syst?me
on observe que la deuxi?me ligne est deux fois de la premi?re L2=2L1 alors C est singuli?re
la matrice est non inversible
```

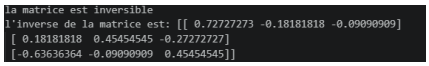
E. Fonction réutilisable

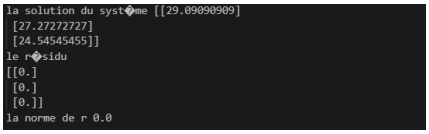
La fonction `analyser_systeme(A,b)` regroupe les principales opérations : calcul du déterminant, vérification de l'inversibilité, calcul de l'inverse, résolution du système, calcul du résidu et de sa norme.
Elle peut être appelée avec :

`analyser_systeme(A,b)`

Elle permet ainsi de réutiliser les mêmes opérations pour l'analyse d'autres systèmes linéaires.







Conclusion

Ce mini-projet a permis de mettre en pratique plusieurs notions d'algèbre linéaire avec Python et NumPy : manipulation des matrices, déterminant, inverse et résolution d'un système linéaire.

On a également comparé deux méthodes de résolution et vérifié le résultat avec `numpy.allclose` et le résidu. Enfin, l'étude de la matrice singulière a permis de mettre en évidence le lien entre déterminant nul, dépendance linéaire et absence d'inverse.